

Análisis Reto: GIRO

Sistema de Gestión de Inventario y Pedidos

Hansell Steven Benavides Jumbo

1 de junio de 2026

Índice general

1. Capítulo 1: Introducción y Problemática	2
1.1. Introducción	2
1.2. Problemática	2
1.3. Metodología o estrategia de desarrollo	2
2. Capítulo 2: Planificación y análisis	3
2.1. Análisis	3
2.2. Requisitos funcionales	3
2.3. Requisitos no funcionales	3
2.4. Historias de Usuario / Casos de Uso	4
2.5. Sprint	7
2.6. Diagramas: Clases y BD (diseño lógico)	8
2.7. Diagrama de componentes	9
2.8. Arquitectura lógica y física (propuesta)	9
2.9. Prototipo de pantallas no funcional	10

Capítulo 1

Capítulo 1: Introducción y Problemática

1.1. Introducción

El presente documento tiene como propósito definir la visión general del Sistema de Gestión de Inventario y Rutas de Pedido, denominado GIRO. Este sistema se establece en el contexto de la distribución mayorista, abarcando la atención de micromercados en zonas tanto rurales como urbanas del Ecuador. Su objetivo principal es reducir la pérdida de ventas, mejorar el control de inventario en tiempo real y potenciar la productividad de los vendedores en ruta.

1.2. Problemática

La problemática central radica en la desconexión entre el inventario real disponible en bodega y la información que manejan los vendedores en campo al momento de tomar pedidos en micromercados. Los vendedores trabajan con catálogos desactualizados o con datos de stock del día anterior, lo que genera pedidos sobre productos agotados, compromisos de entrega que no se pueden cumplir y ventas perdidas. Esto impacta negativamente a los vendedores, jefes de ventas, bodega, propietarios de negocios y a la gerencia general de la distribuidora.

1.3. Metodología o estrategia de desarrollo

La estrategia de desarrollo se apoya en iteraciones ágiles, validación continua con stakeholders y la ejecución de *sprints* de refinamiento junto al equipo de desarrollo. Se incorporan también prácticas de análisis de negocio basadas en estándares del BABOK.

A nivel de arquitectura técnica, el proyecto utilizará **MariaDB** como motor de base de datos relacional. Se seleccionó MariaDB por su alta escalabilidad, rendimiento en la nube y la facilidad de implementación con el equipo actual. Para el desarrollo del backend, se empleará el framework **Django** (Python), el cual nos permitirá desarrollar de forma rápida, estructurada y segura la API REST que servirá los datos tanto a la aplicación web (PWA) de los vendedores como al panel de bodega.

Capítulo 2

Capítulo 2: Planificación y análisis

2.1. Análisis

El análisis revela que una solución exitosa permitirá la consulta de stock en tiempo real, el registro digital de pedidos en campo mediante una aplicación web, la aplicación de incentivos dinámicos y la visibilidad gerencial mediante paneles centralizados. El enfoque se centra en vendedores en ruta (usuarios principales) y supervisores.

2.2. Requisitos funcionales

A continuación, se listan los requisitos funcionales principales extraídos del análisis:

- **RF-01:** El sistema debe mostrar al vendedor un catálogo de productos con el nombre, código SKU, descripción, precio unitario y nivel de stock actualizado.
- **RF-02:** El sistema debe permitir al vendedor seleccionar un cliente de su ruta e iniciar un nuevo pedido.
- **RF-03:** El sistema debe operar en modo offline almacenando localmente el catálogo y los pedidos generados, sincronizando automáticamente al recuperar la señal.
- **RF-04:** Al confirmar un pedido, el sistema debe verificar que la cantidad solicitada no supere el stock disponible.
- **RF-05:** El sistema debe actualizar el nivel de inventario automáticamente al confirmar el despacho de un pedido.
- **RF-06:** El sistema debe mostrar al supervisor un panel con el estado de pedidos por vendedor y el cumplimiento de metas en tiempo real.

2.3. Requisitos no funcionales

Los requisitos de calidad técnica y restricciones operativas incluyen:

- **RNF-01 (Rendimiento):** El catálogo debe cargar completamente en el navegador del vendedor en menos de 3 segundos con una conexión 3G estable.

- **RNF-02 (Rendimiento):** La sincronización de pedidos offline debe completarse en menos de 10 segundos al recuperar la conexión.
- **RNF-03 (Seguridad):** Toda comunicación debe realizarse mediante HTTPS con TLS 1.3.
- **RNF-04 (Usabilidad):** El vendedor debe poder completar la toma de un pedido en un máximo de 5 pasos desde la aplicación web en su dispositivo móvil.
- **RNF-05 (Disponibilidad):** Garantizar un uptime mínimo del 99.5 % mensual.

2.4. Historias de Usuario / Casos de Uso

En esta sección se modela la interacción entre los actores del sistema GIRO y las funcionalidades principales mediante diagramas de casos de uso.

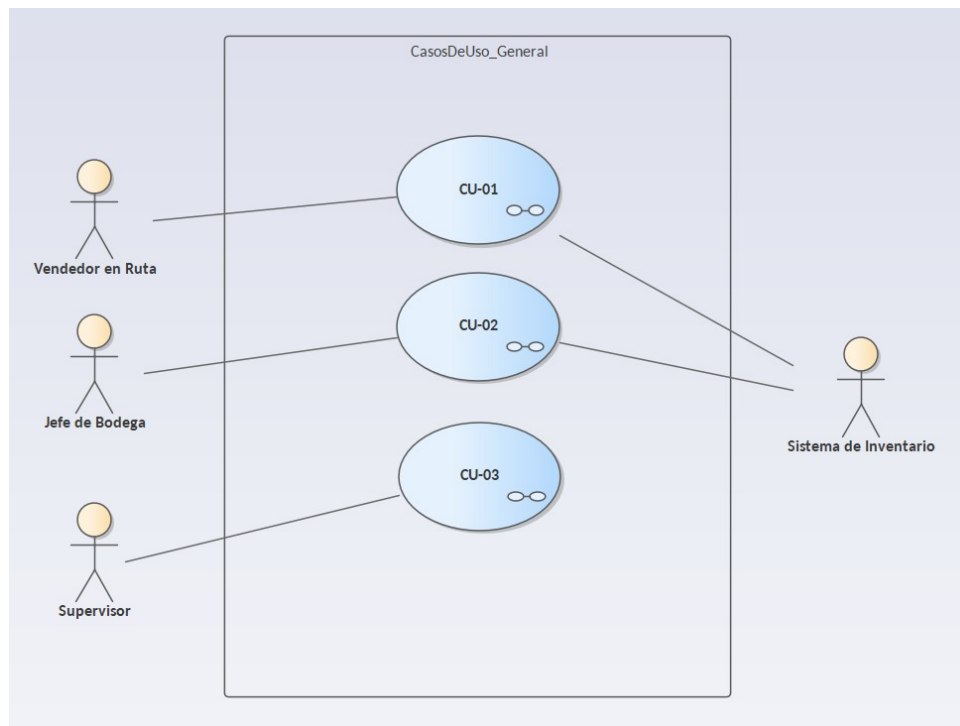


Figura 2.1: Diagrama de Casos de Uso General

En la Figura 2.1 se ilustra la visión global del sistema. Se identifican los tres actores principales (Vendedor en Ruta, Jefe de Bodega y Supervisor) interactuando con los módulos centrales del sistema (CU-01, CU-02 y CU-03), apoyados de fondo por el sistema general de inventario.

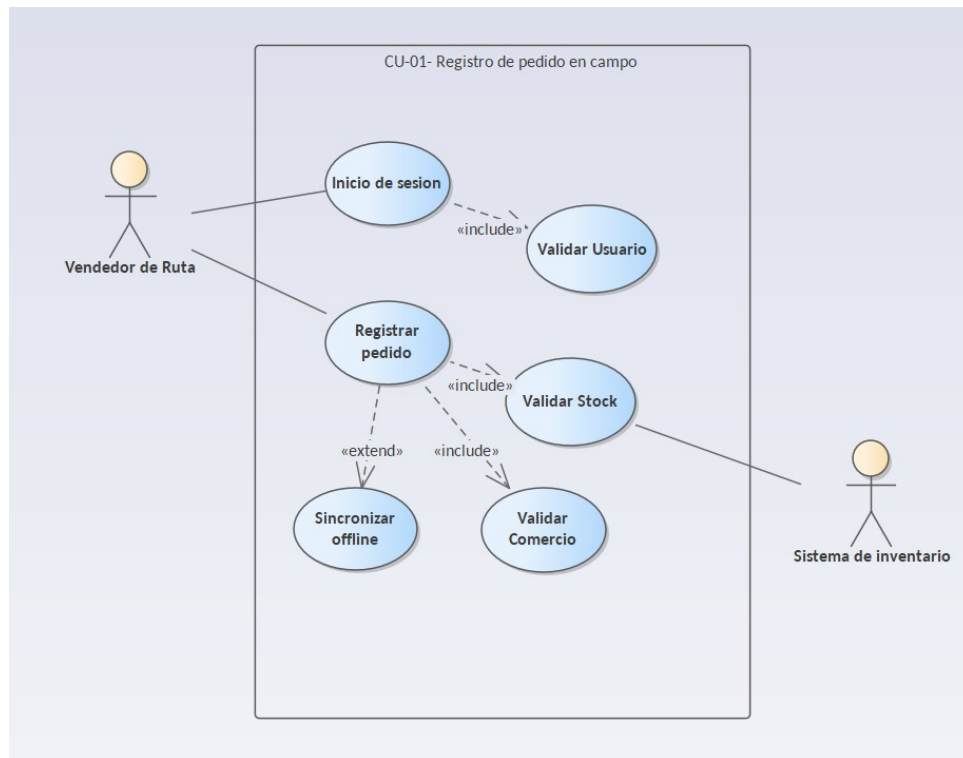


Figura 2.2: Flujo Principal: Registro de pedido en campo

La Figura 2.2 detalla el Caso de Uso 01 (Registro de pedido). Aquí se observa cómo el Vendedor de Ruta inicia el proceso con un inicio de sesión (que incluye la validación del usuario) y procede a registrar el pedido.

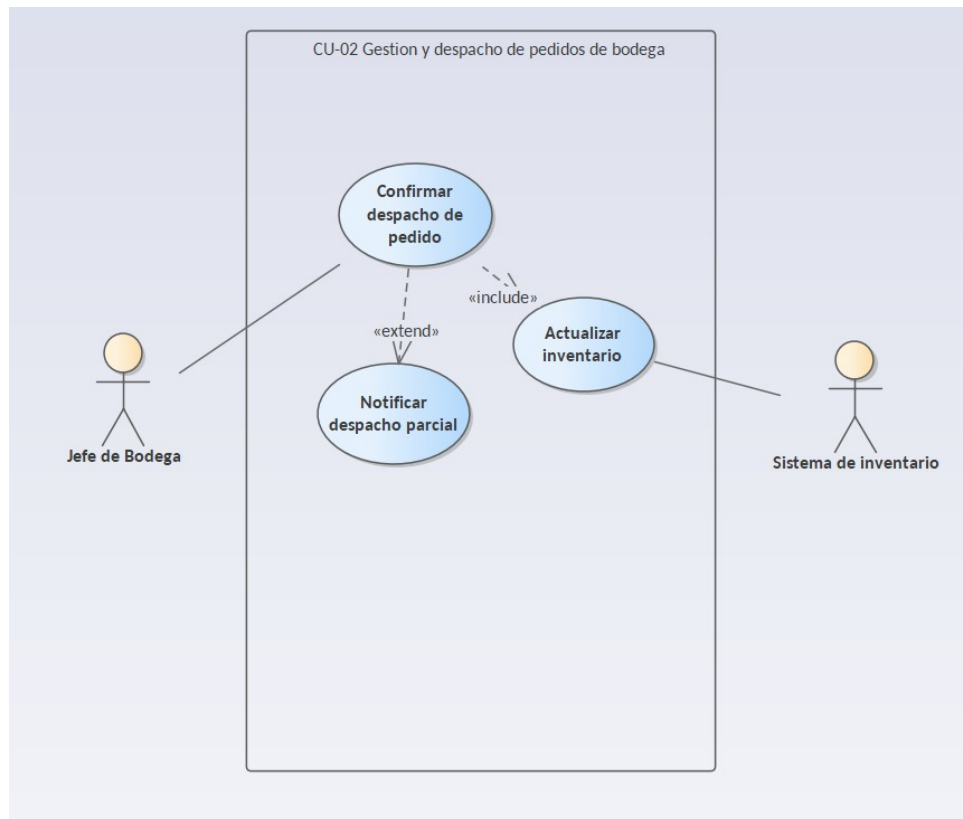


Figura 2.3: Validaciones del Registro de Pedido

Como se detalla en la Figura 2.3, el registro del pedido tiene dependencias estrictas (*includes*): requiere validar el stock en tiempo real contra el sistema de inventario y validar que el comercio pertenezca a la ruta del vendedor.

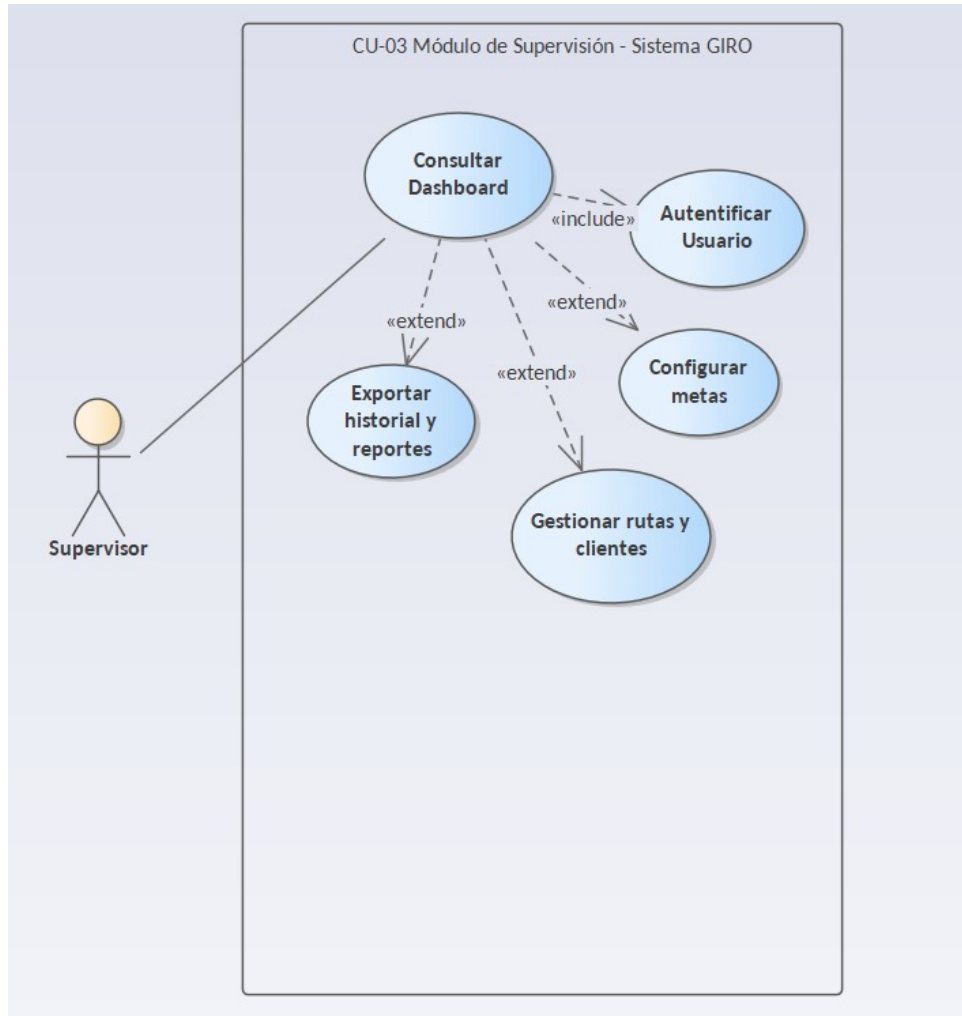


Figura 2.4: Extensión Offline del Registro de Pedido

Finalmente, la Figura 2.4 resalta la funcionalidad *extend* del sistema: en caso de que el vendedor se encuentre en una zona rural sin cobertura, el sistema extiende su comportamiento para habilitar la sincronización offline, guardando el pedido localmente.

2.5. Sprint

La estrategia de desarrollo se divide en iteraciones o *Sprints* enfocados en entregar valor incremental alineado a los módulos críticos del sistema:

- **Sprint 1: Base, Autenticación y Catálogo.** Configuración de la base de datos, despliegue de la API, módulo de seguridad con control de acceso basado en roles (RBAC) y listado de productos en tiempo real.
- **Sprint 2: Módulo Web de Campo y Sincronización.** Desarrollo de la Progressive Web App (PWA) para vendedores y lógica de toma de pedidos con almacenamiento local cifrado (AES-256) para habilitar el funcionamiento offline en zonas rurales sin cobertura.
- **Sprint 3: Bodega y Supervisión.** Implementación del panel de control web en

tiempo real para supervisores, gestión de metas dinámicas y la cola de despacho para el equipo de bodega.

2.6. Diagramas: Clases y BD (diseño lógico)

A continuación, se presenta el diseño estructural del sistema, modelado en dos niveles: el diseño orientado a objetos y el diseño físico para la base de datos.

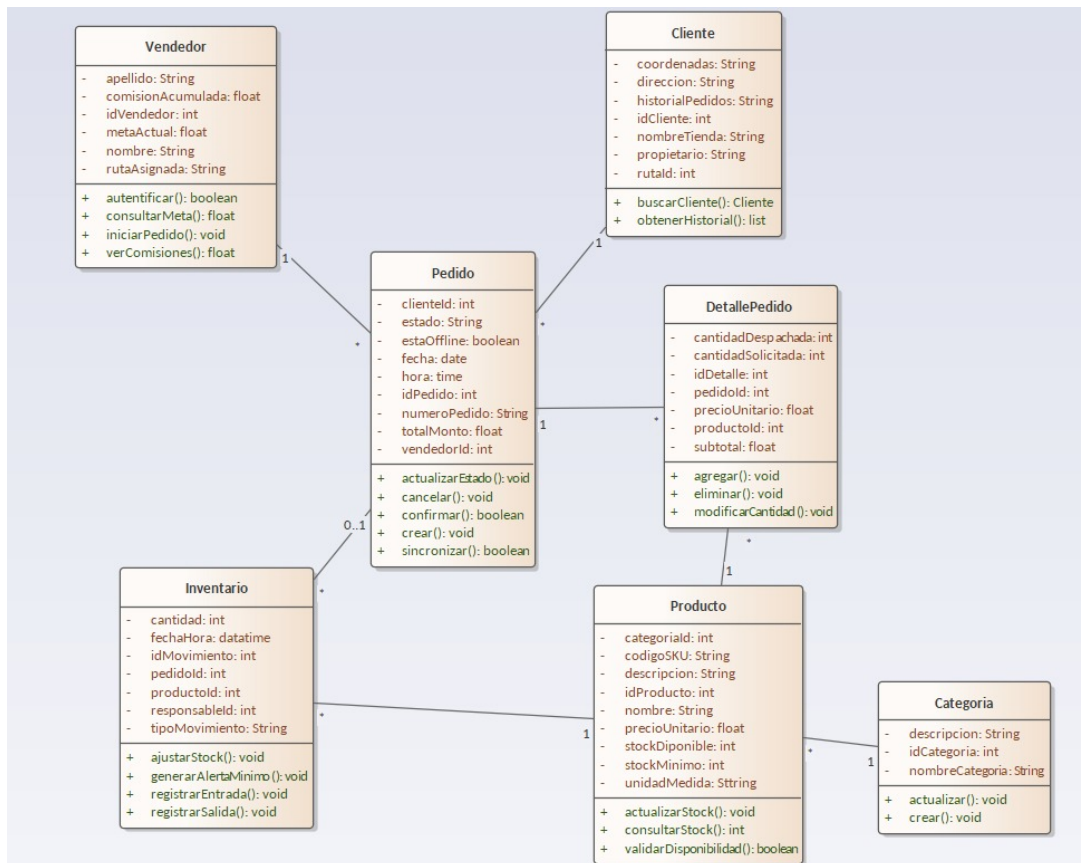


Figura 2.5: Diagrama de Clases (Lógica de Negocio)

La Figura 2.5 muestra el diagrama de clases de UML. Este modelo define las entidades principales (Vendedor, Cliente, Pedido, Producto, etc.), sus atributos y sus métodos de operación, estableciendo las relaciones y multiplicidades lógicas (por ejemplo, la relación de uno a muchos entre un vendedor y sus pedidos).

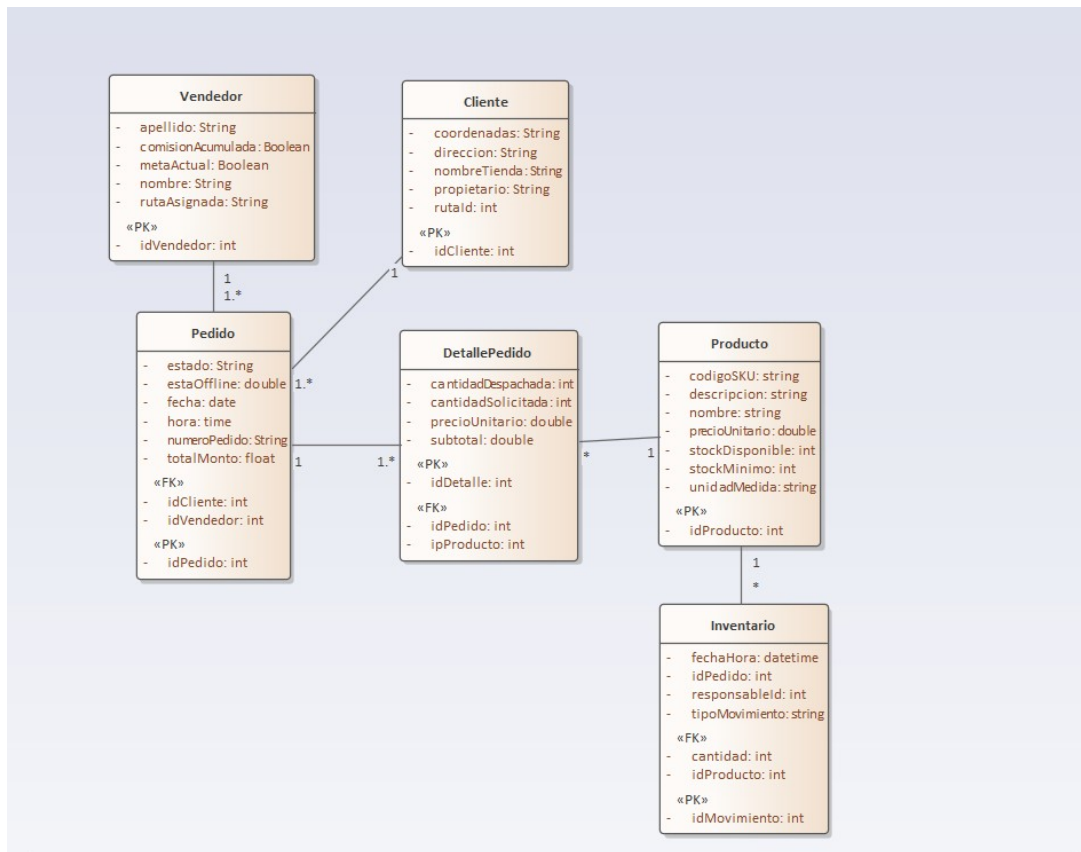


Figura 2.6: Diagrama Lógico de Base de Datos para MariaDB

La Figura 2.6 representa la estructura adaptada específicamente para el motor relacional **MariaDB**. En este diagrama se visualizan los tipos de datos físicos y se identifican explícitamente las Llaves Primarias («PK») de cada tabla, así como las Llaves Foráneas («FK») que garantizan la integridad referencial y el intercambio de datos entre los distintos módulos del sistema.

2.7. Diagrama de componentes

El sistema se divide en tres componentes principales: la aplicación web (PWA) en los dispositivos de los vendedores, el servidor que expone la API REST, y el motor de base de datos relacional.

2.8. Arquitectura lógica y física (propuesta)

El sistema GIRO adopta una arquitectura orientada a servicios, diseñada para alta disponibilidad y el manejo de contextos con baja conectividad:

- **Arquitectura Lógica:** El frontend opera como una aplicación web progresiva (PWA) con capacidades *offline-first*, sincronizando datos asíncronamente con el backend. Este último expone una API REST que centraliza la lógica de validación de stock y el cálculo de comisiones.

- **Arquitectura Física:** Los usuarios acceden mediante navegadores web desde smartphones o computadoras de escritorio. El servidor web y la base de datos se alojan en una infraestructura en la nube que permite el escalamiento horizontal, garantizando un *uptime* del 99.5 %.

2.9. Prototipo de pantallas no funcional

Las interfaces web están diseñadas con un enfoque responsivo para adaptarse a pantallas móviles y de escritorio, cumpliendo con la restricción de diseño de botones táctiles de 44x44 píxeles para el uso en campo.